

```
"""
```

app.py – Saree Virtual Try-On API

A small Flask API with exactly two inputs required from the customer:

1. customer_photo – a photo of the customer
2. saree_photo – a photo of the saree

No text prompt is required or accepted anywhere. The output image is produced purely by pose detection + geometric warping + compositing (see tryon_engine.py) – there is no generative/diffusion model in the loop, so the saree's actual pattern, colour and texture in the output always come directly from the photo the customer/shop uploaded.

Run locally:

```
pip install flask opencv-python-headless mediapipe pillow numpy
python app.py
# server starts on http://0.0.0.0:5000
```

Endpoints

```

GET /api/health
-> {"status": "ok"}

POST /api/tryon
    multipart/form-data body:
        customer_photo : image file
(jpg/png)
        saree_photo    : image file
(jpg/png)
    optional form fields:
        drape_width_factor : float, default
1.18
        feather            : int,
default 21
    -> returns image/png bytes of the
composited result

    On failure, returns JSON: {"error":
"..."} with an appropriate
    HTTP status code (400 for bad input,
422 if no person is detected,
    500 for unexpected errors).
"""

from __future__ import annotations

import io
import traceback

from flask import Flask, request, jsonify,
send_file, send_from_directory

from tryon_engine import virtual_tryon,
NoPersonDetectedError, ModelNotFoundError

```

```
app = Flask(__name__,
static_folder="static", static_url_path="")

@app.get("/")
def index():
    return
send_from_directory(app.static_folder,
"index.html")

ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png",
"webp"}
MAX_UPLOAD_MB = 15
app.config["MAX_CONTENT_LENGTH"] =
MAX_UPLOAD_MB * 1024 * 1024

def _allowed_file(filename: str) -> bool:
    return "." in filename and
filename.rsplit(".", 1)[1].lower() in
ALLOWED_EXTENSIONS

@app.get("/api/health")
def health():
    return jsonify({"status": "ok"})

@app.post("/api/tryon")
def tryon():
    if "customer_photo" not in
request.files or "saree_photo" not in
request.files:
```

```

        return jsonify({
            "error": "Both 'customer_photo'
and 'saree_photo' files are required."
        }), 400

        customer_file =
request.files["customer_photo"]
        saree_file =
request.files["saree_photo"]

        if customer_file.filename == "" or
saree_file.filename == "":
            return jsonify({"error": "One or
both files were empty."}), 400

        if not
(_allowed_file(customer_file.filename) and
_allowed_file(saree_file.filename)):
            return jsonify({
                "error": f"Only these file
types are allowed:
{sorted(ALLOWED_EXTENSIONS)}"
            }), 400

        try:
            drape_width_factor =
float(request.form.get("drape_width_factor"
, 1.18))
            feather =
int(request.form.get("feather", 21))
        except ValueError:
            return jsonify({"error":
"drape_width_factor/feather must be
numeric."}), 400

```

```

        try:
            customer_bytes =
customer_file.read()
            saree_bytes = saree_file.read()

            result_png = virtual_tryon(
                customer_bytes,
                saree_bytes,

drape_width_factor=drape_width_factor,
                feather=feather,
            )
        except NoPersonDetectedError as e:
            return jsonify({"error": str(e)}),
422
        except ModelNotFoundError as e:
            return jsonify({"error": str(e)}),
500
        except Exception as e: # noqa: BLE001
            app.logger.error("tryon failed:
%s\n%s", e, traceback.format_exc())
            return jsonify({"error": "Internal
error while generating try-on image."}),
500

    return send_file(
        io.BytesIO(result_png),
        mimetype="image/png",
        as_attachment=False,
        download_name="tryon_result.png",
    )

```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000,  
    debug=True)
```